

SWT

CE

The Processor

**Computer
Architecture**

**Michael G. Wahl
Page 1**

The processor

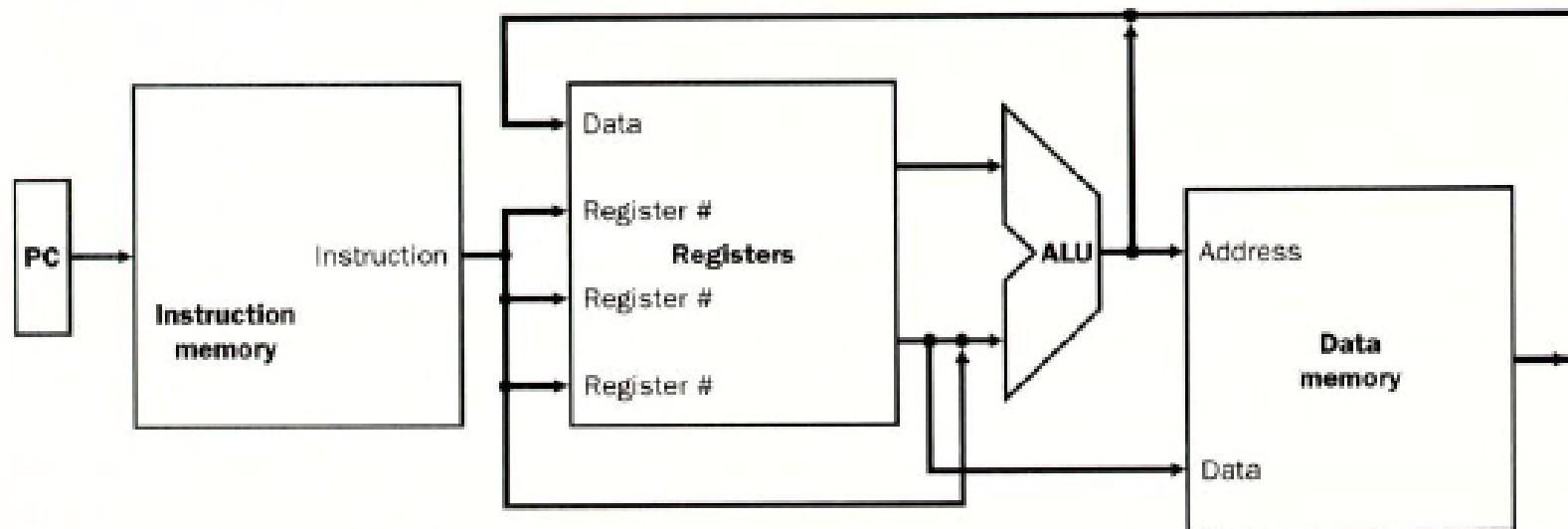
Data path and control

Introduction

- **Building blocks of the processor**
- **Implementation of a subset**
 - ◆ Load & store word
 - ◆ Arithmetic & logic instructions (add, sub, and, or, slt)
 - ◆ Branch equal & jump
- **Logic and clocking**
 - ◆ Edge triggered
 - ◆ New value at the rising edge of the clock
 - ◆ Period determined by the longest module
 - ◆ Registers: automatically updated, except those with an explicit write signal

The abstract view

- Program counter: next instruction
- Instruction memory
- Register file
- Data memory
- Bus structure: information transfer



Building the data path

■ Instruction memory

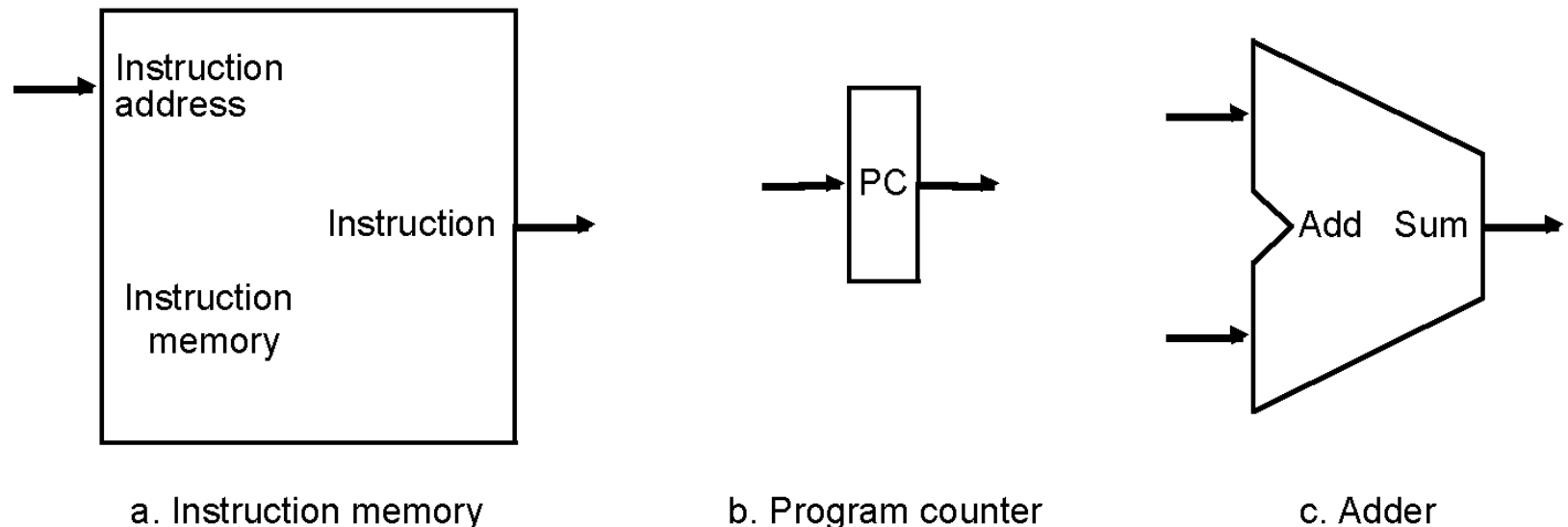
- ◆ Contains **only** instructions

■ Program counter

- ◆ points to the next instruction

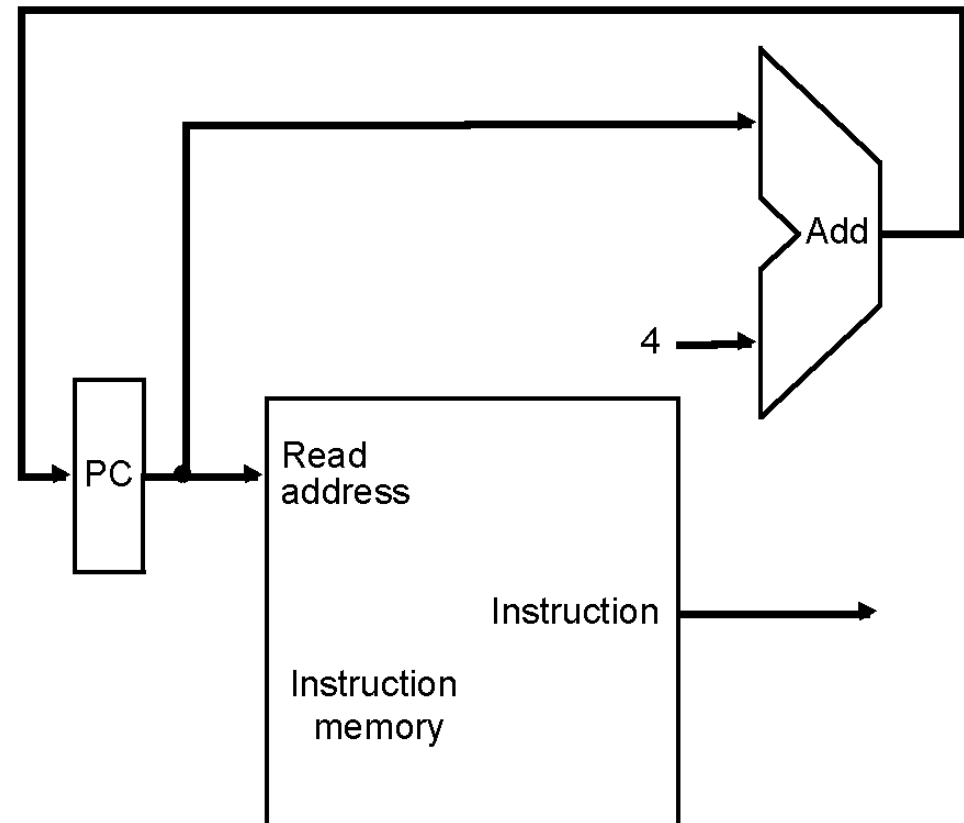
■ Arithmetic & logic unit

- ◆ Here: an adder only



Instructions & PC

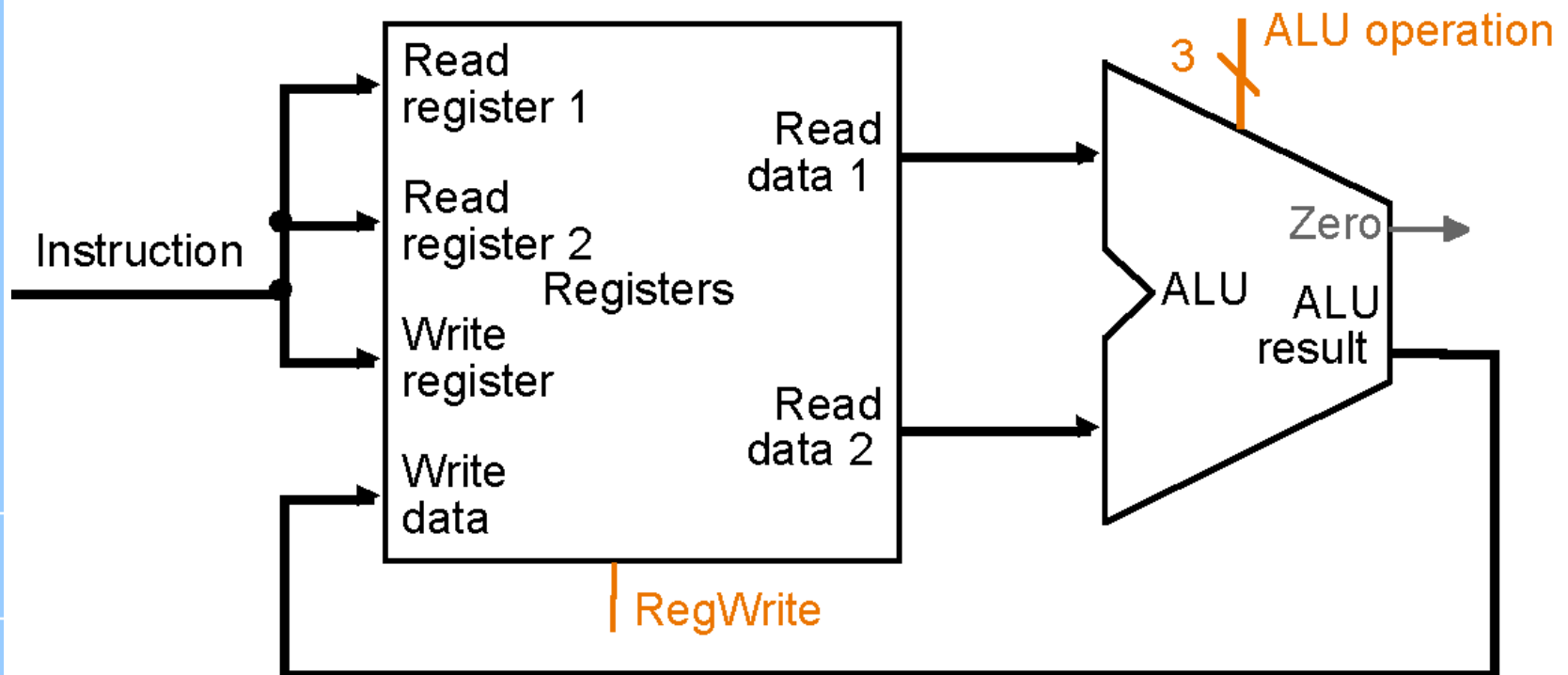
- **Increment:**
add 4 to
the PC to
get to the
address of
the next
instruction
- **Needs
extension
for Branch
and Jump**



Registers and ALU

- Register file for R-format instructions as discussed previously
- ALU as designed in the previous chapter

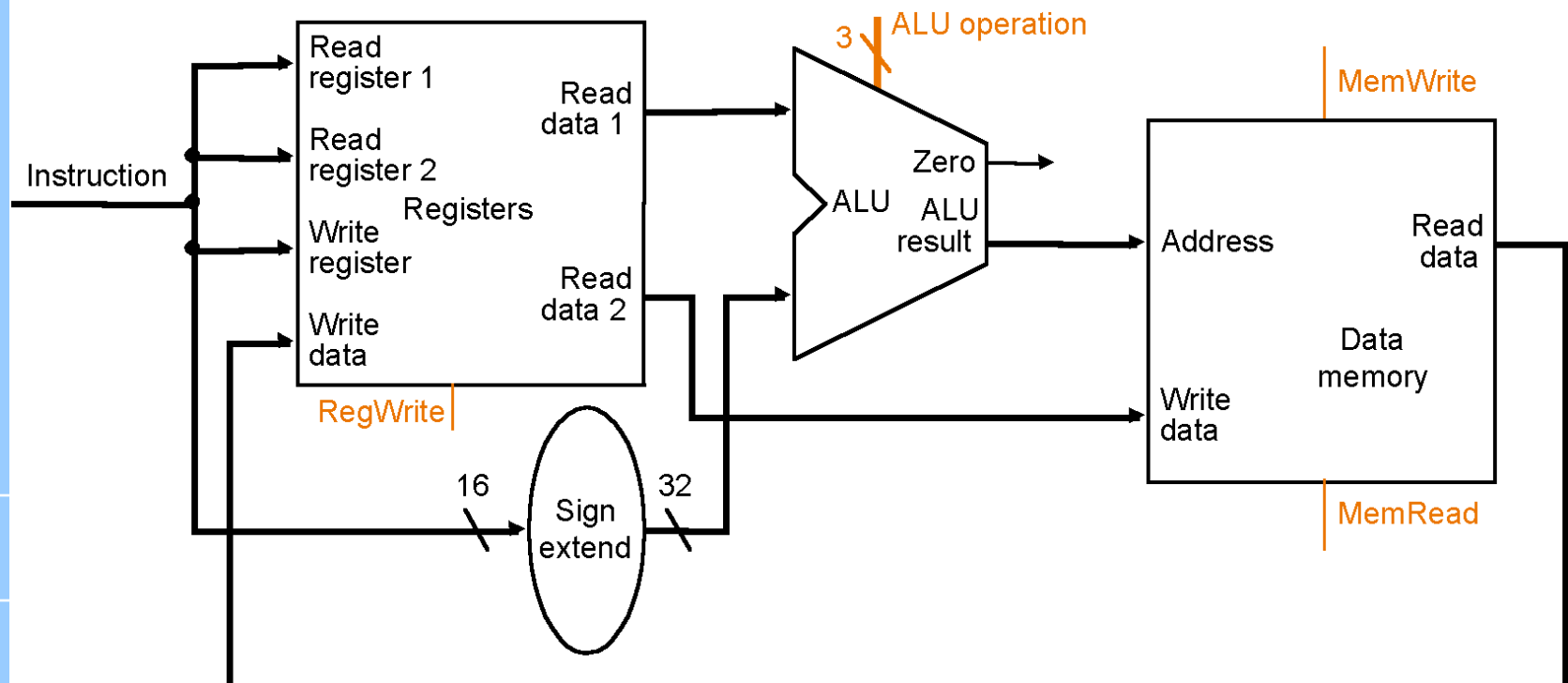
The Processor



Adding the data memory

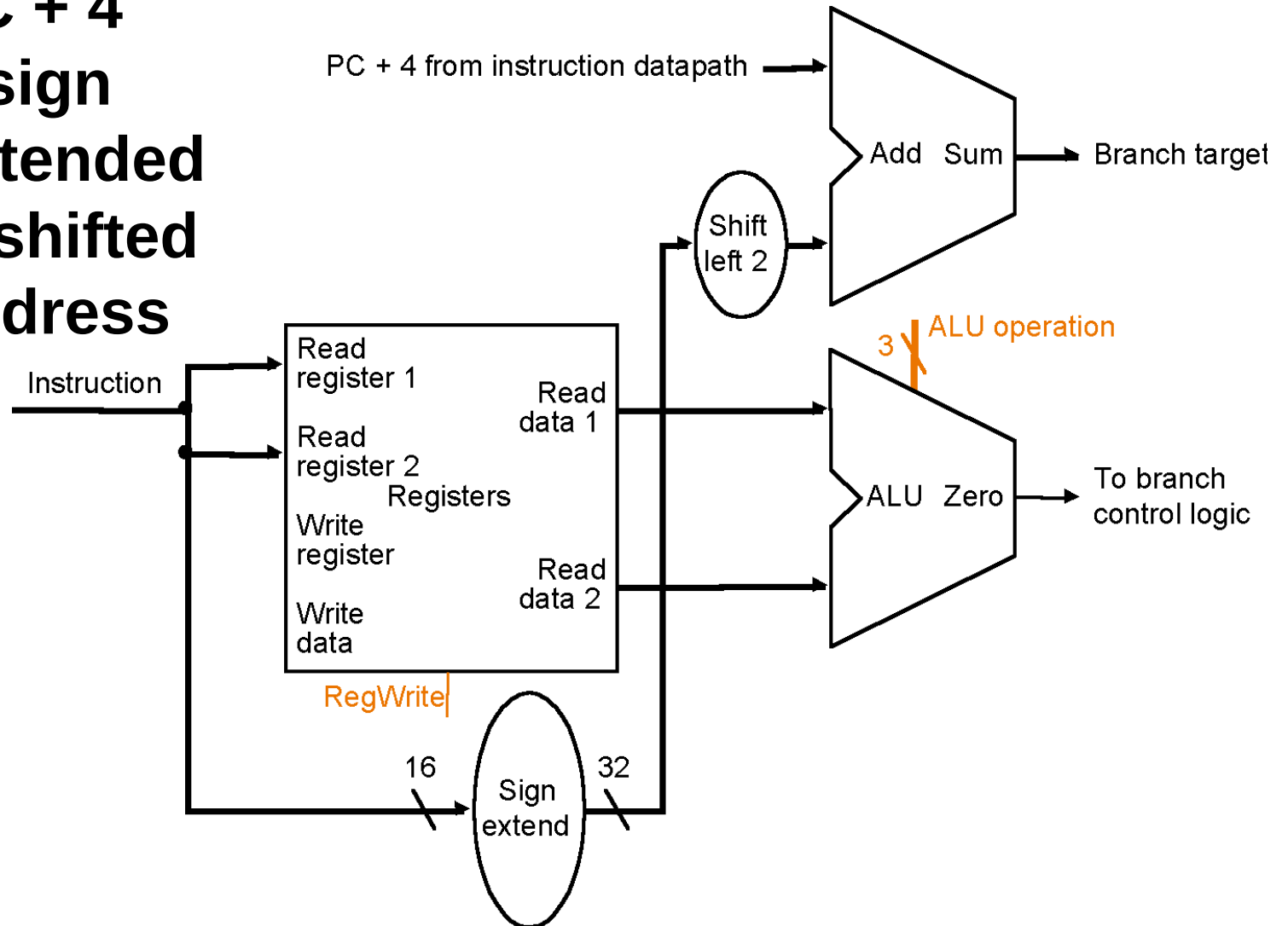
- Transfer address to the memory
- Transfer data to the register file
- Sign extension unit for address calculation

The Processor



Branch data path

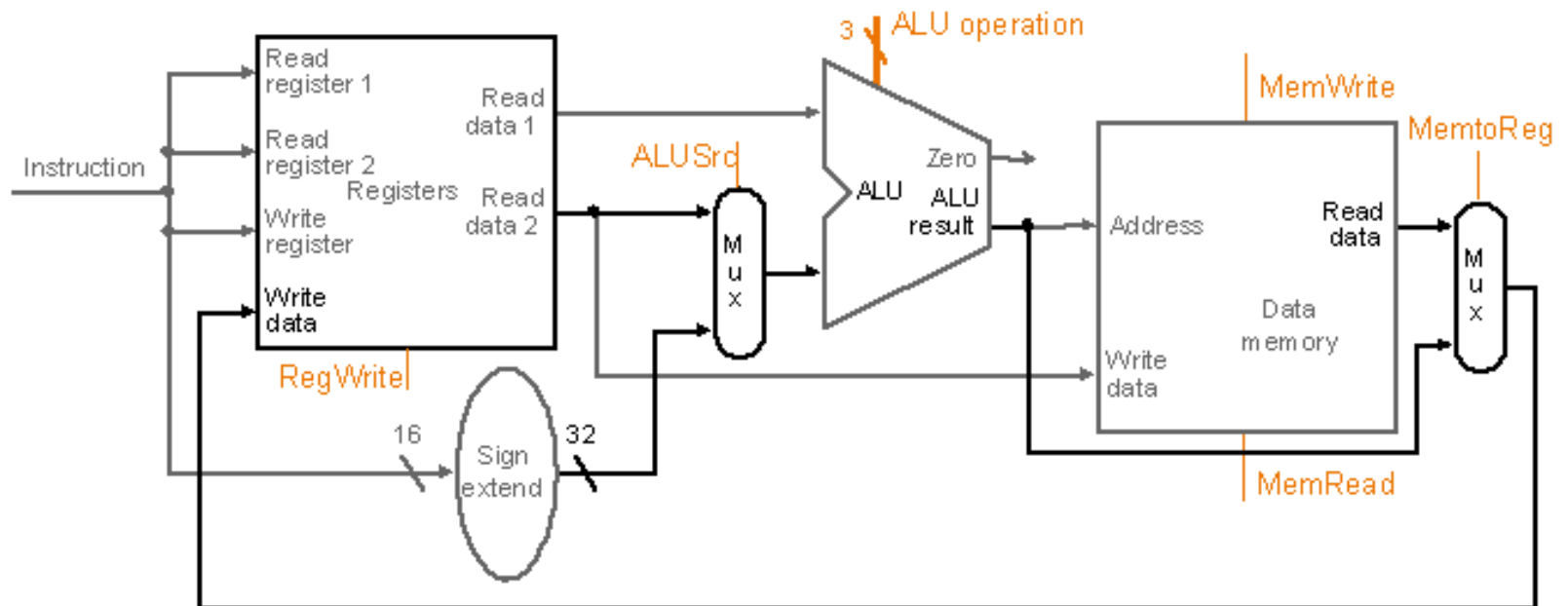
■ PC + 4
+ sign
extended
& shifted
address



Putting things together

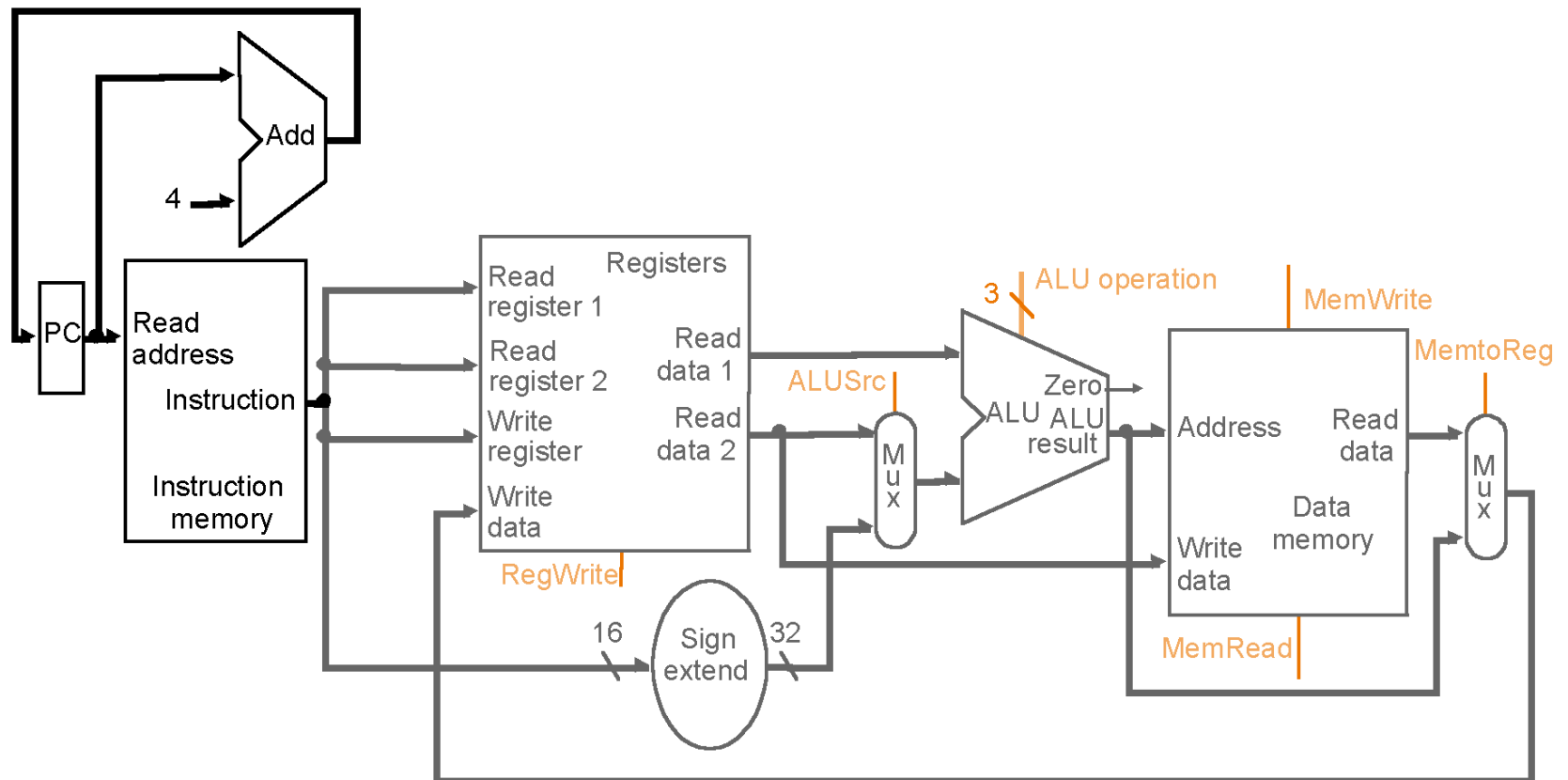
- Register set
- ALU for operand and address calculation
- Data memory

The Processor



Adding the instruction fetch unit

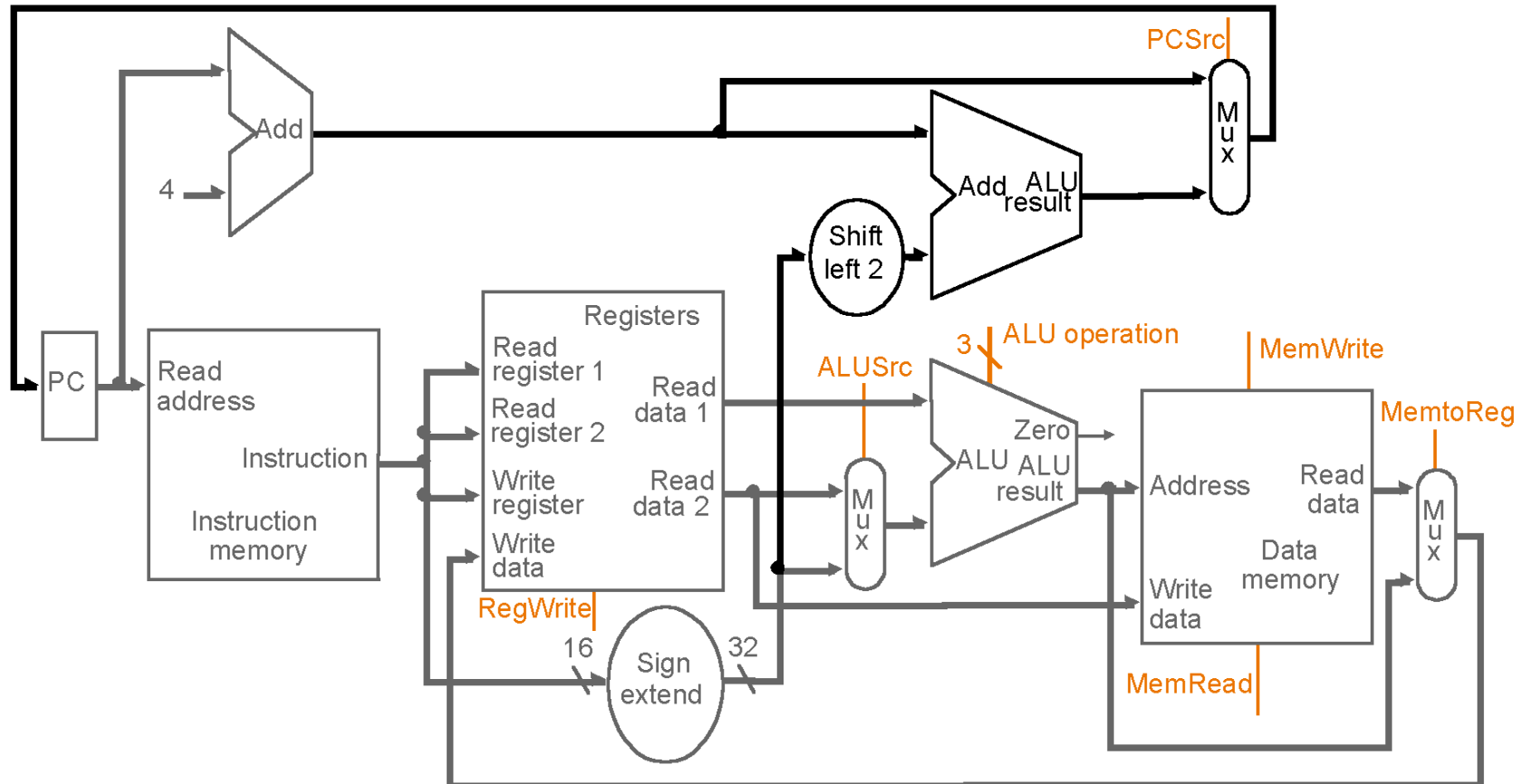
- The instruction fetch unit provides the instruction



Adding branch logic

- Supports now all basic instructions

The Processor



Basic control

■ Review of ALU functions / control lines

ALU Control lines	Function
000	And
001	Or
010	Add
110	Subtract
111	Set-on-less-than

■ ALU control bits: origin of

- ◆ 00: Addition (load & store word)
- ◆ 01 : Subtraction (branch on equal)
- ◆ 10 : Operations according the value in the function field (R-type instructions)

ALU control

Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	010
SW	00	store word	XXXXXX	add	010
Branch equal	01	branch equal	XXXXXX	subtract	110
R-type	10	add	100000	add	010
R-type	10	subtract	100010	subtract	110
R-type	10	AND	100100	and	000
R-type	10	OR	100101	or	001
R-type	10	set on less than	101010	set on less than	111

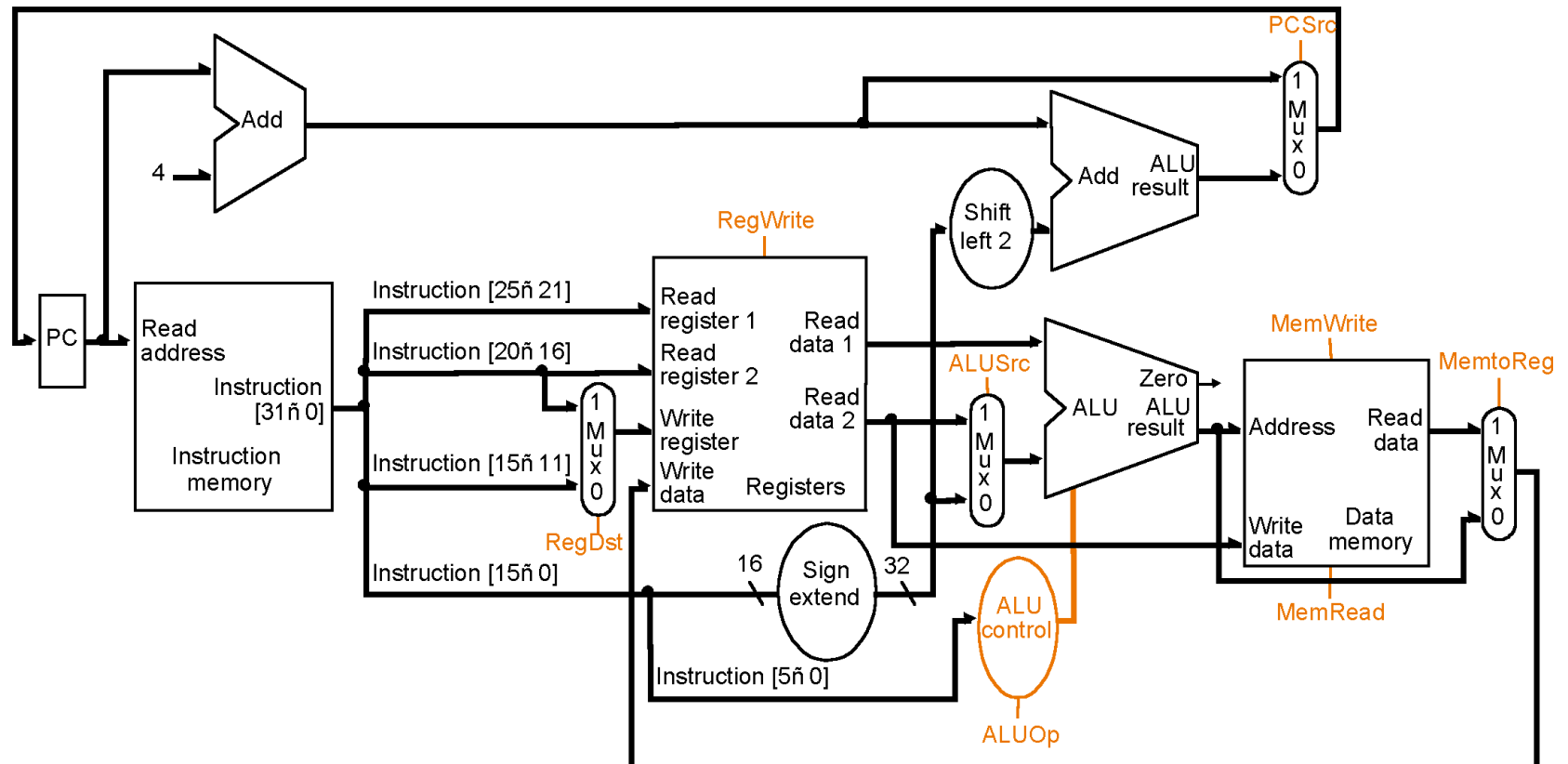
ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	010
X	1	X	X	X	X	X	X	110
1	X	X	X	0	0	0	0	010
1	X	X	X	0	0	1	0	110
1	X	X	X	0	1	0	0	000
1	X	X	X	0	1	0	1	001
1	X	X	X	1	0	1	0	111

Main control

- Source information: instruction
- Operation code: Op [31-26]
- Read registers: rs [25-21], rt [20-16]
- Base register (LW, SW): rs [25-21]
- Destination register
 - ◆ Load: rt [20-16]
 - ◆ R-type: rd [15-11]
 - ◆ Requires a multiplexor

Extended data path

- Multiplexors for write register select
- All control lines

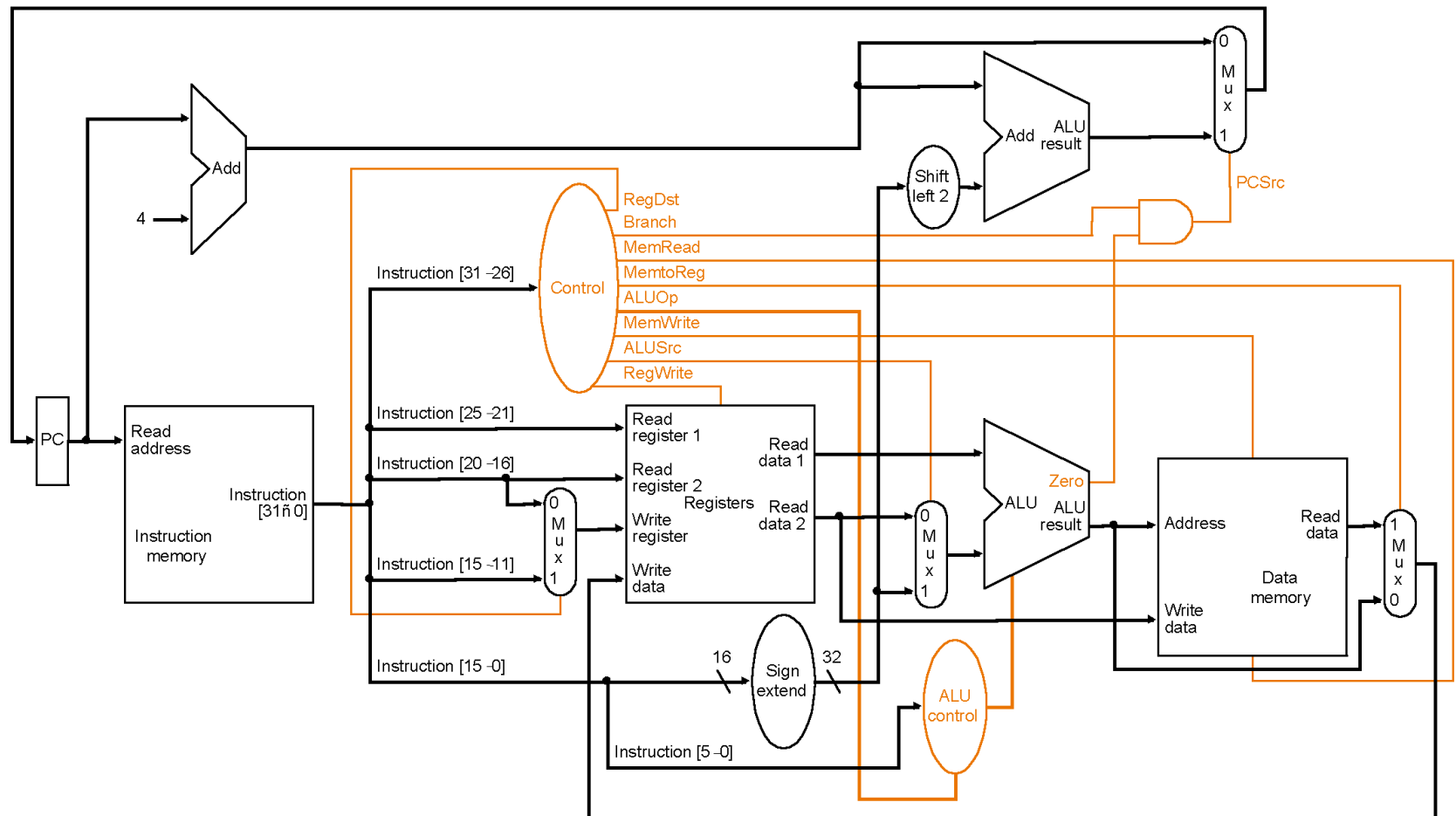


Summary of control lines

- ◆ **RegDest**
Source of the destination register for the operation
- ◆ **RegWrite**
Enables writing a register in the register file
- ◆ **ALUsrc**
Source of second ALU operand, can be a register or part of the instruction
- ◆ **PCsrc**
Source of the PC (increment [POC + 4] or branch)
- ◆ **MemRead / MemWrite**
Reading / Writing from memory
- ◆ **MemtoReg**
Source of write register contents

Data path & control

The Processor



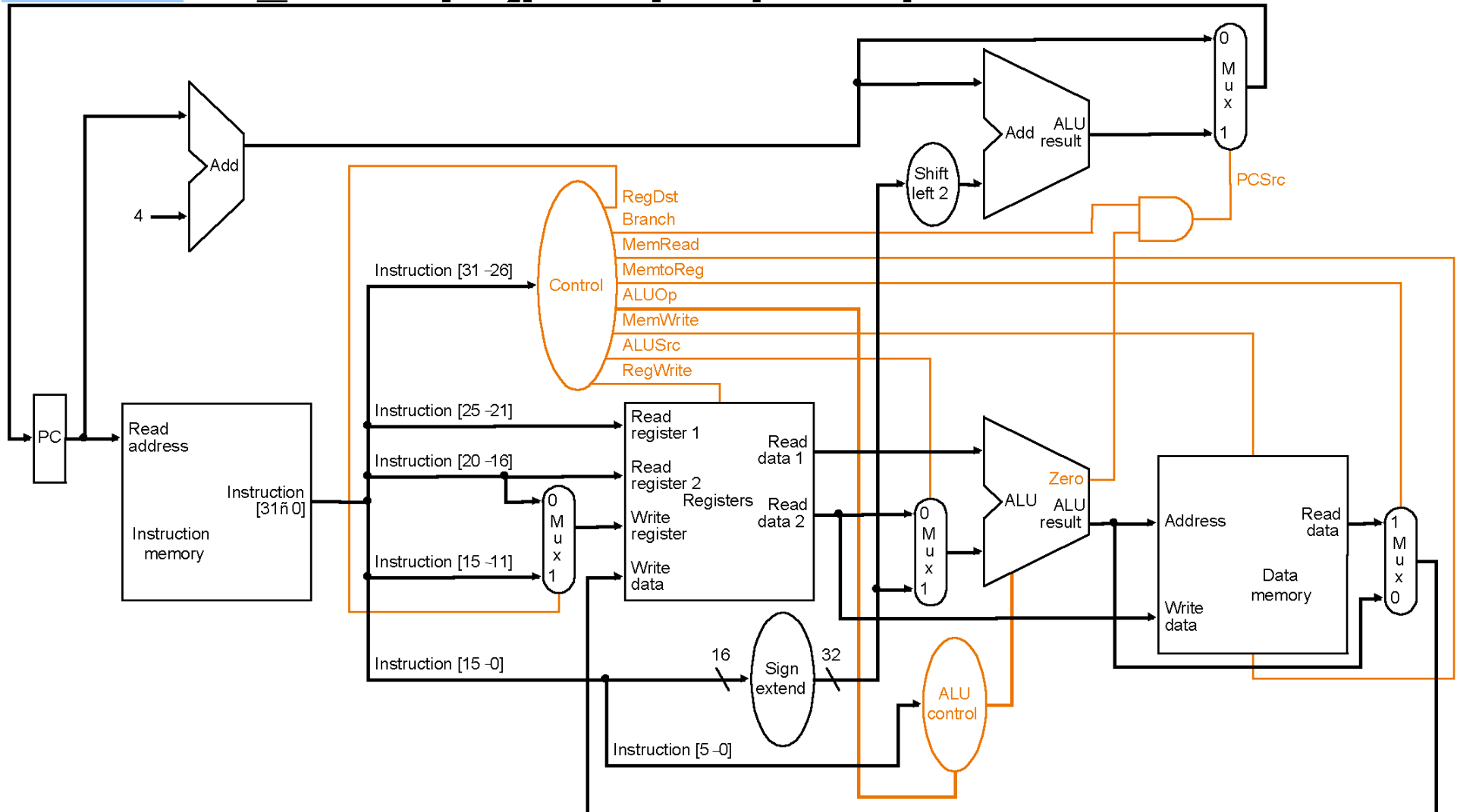
Operation of data path

Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

■ Example flow: R-type instruction

- ◆ Instruction fetch and PC increment
- ◆ Reading of the registers
- ◆ Processing of the data in the ALU
- ◆ Writing the result to the register file

Example flow: R-type instruction



Example flow: load word

■ lw \$3, offset (\$2)

- ◆ Fetch instruction from memory, increment PC
- ◆ Read register value from register \$2
- ◆ Compute the final address
- ◆ Use address for for addressing the memory
- ◆ Write the data into the register file

■ beq \$10, \$11, offset

- ◆ Fetch instruction form memory, increment PC
- ◆ Read the two register values (\$10, \$11)
- ◆ Subtract the values, calculate the branch target address
- ◆ Use the zero signal to determine the target

Finalising the control

- Decimal op code
- Binary op code

Name	Opcode in decimal	Opcode in binary					
		Op5	Op4	Op3	Op2	Op1	Op0
R-format	0 _{ten}	0	0	0	0	0	0
lw	35 _{ten}	1	0	0	0	1	1
sw	43 _{ten}	1	0	1	0	1	1
beq	4 _{ten}	0	0	0	1	0	0

Control function

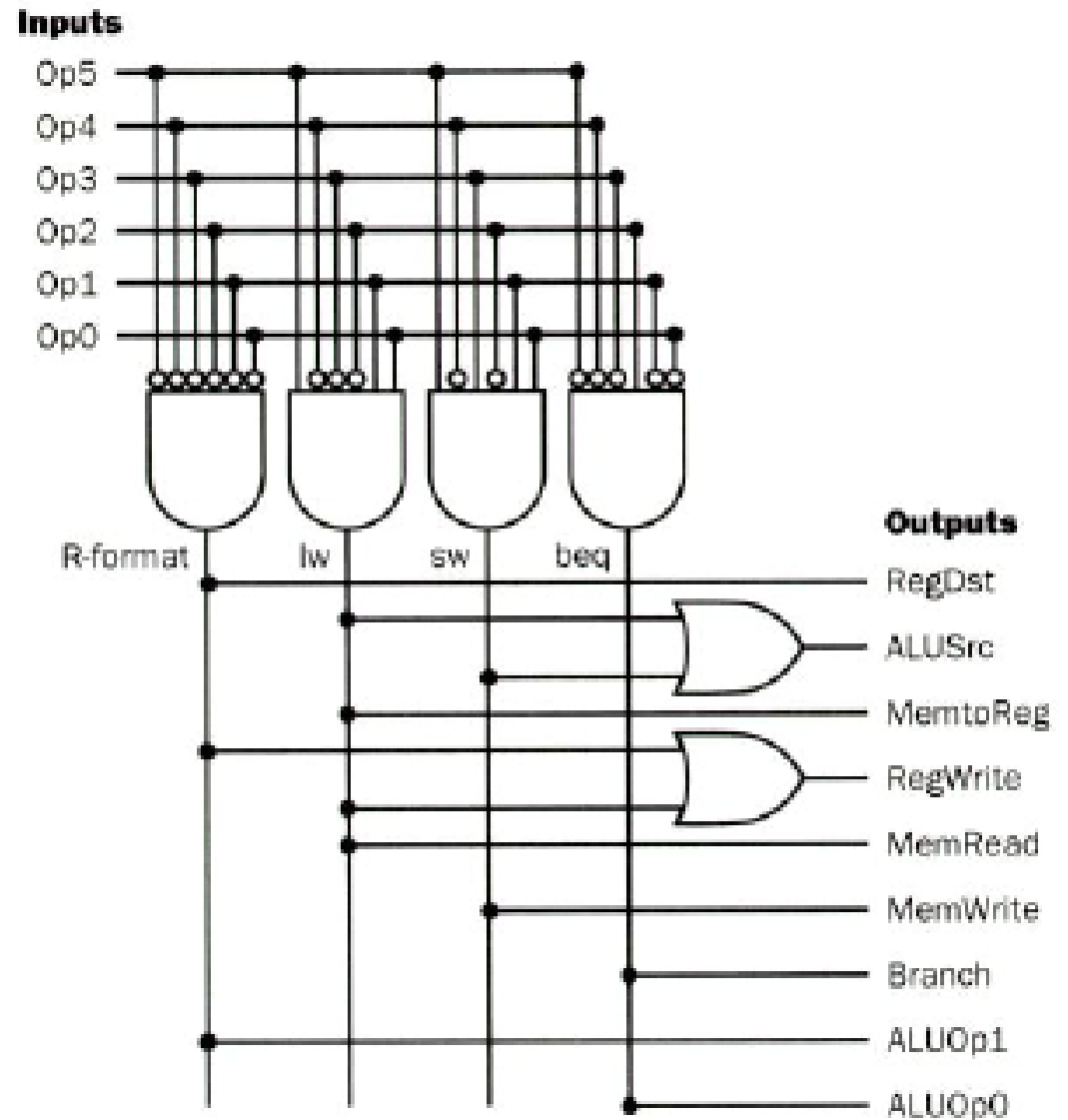
■ Choose the right implementation

R-format		lw	sw	beq
Inputs	Op5	0	1	0
	Op4	0	0	0
	Op3	0	0	1
	Op2	0	0	1
	Op1	0	1	0
	Op0	0	1	0
Outputs	RegDst	1	0	X
	ALUSrc	0	1	0
	MemtoReg	0	1	X
	RegWrite	1	1	0
	MemRead	0	1	0
	MemWrite	0	0	1
	Branch	0	0	1
	ALUOp1	1	0	0
	ALUOp0	0	0	1

Implementation as PLA

■ Alternatives

- ◆ Custom logic
- ◆ PLA
- ◆ ROM
- ◆ ...



Jump instruction

- Extension of the previous architecture
- Jump address
- Lower 2 bits always 0
- Immediate field: bits 2-27
- Current PC: bits 28-31

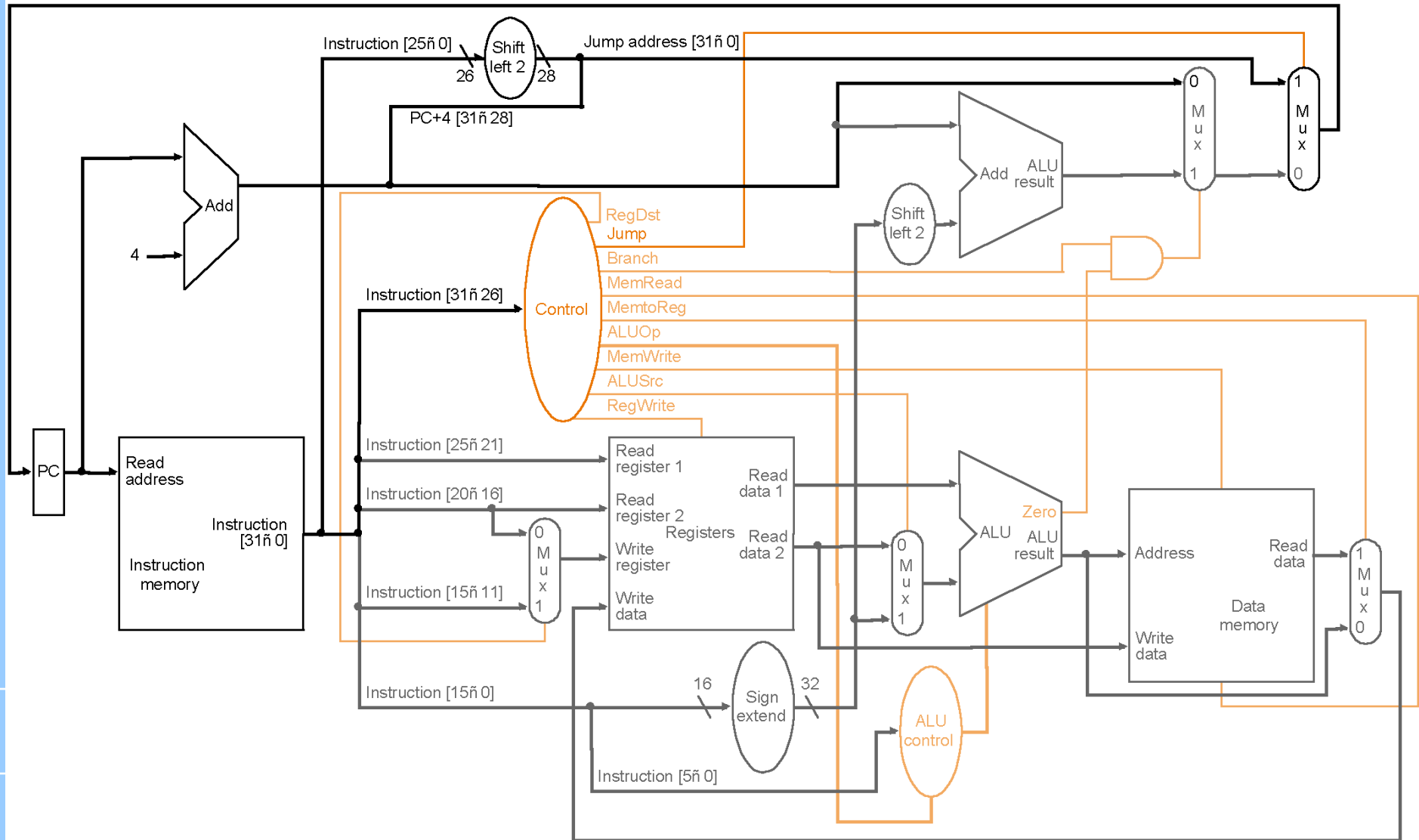
Instruction

op	address
2	80000

Address

PC	jump address field	0

Data path supporting jump



Limitation of single cycle approach

- Access times for various operations vary
 - ◆ Memory access: 10 ns
 - ◆ Register access: 2 ns
 - ◆ Add: 3 ns
 - ◆ Sub: 3.5 ns
 - ◆ Multiplication: 20 ns
 - ◆ FP multiplication: 24 ns
 - ◆ Transcendent functions: 40 ns
- Time depends on implementation
- The slowest module defines the cycle duration

Example

The Processor

- List of actions per instruction
- Number of cycles / instruction
- Faster MIPS

Instruction type	Instruction memory	Register read	ALU operation	Data memory	Register write	Total
R-Format op.	8	2	4	0	2	16
Load word	8	2	4	8	2	24
Store word	8	2	4	8	0	22
Branch	8	2	4	0	0	14
Jump	8	0	0	0	0	8

- Take a benchmark and the probability of instructions

Example

■ Probabilities

◆ R-format	49%	16ns
◆ Load word	22%	24ns
◆ Store word	11%	22ns
◆ Branch	16%	14ns
◆ Jump	2%	8ns

■ Average CPU clock cycle time for this benchmark: 18 ns

■ Clock cycle time by slowest block: 24 ns (ignoring really long operations)